Limitless Deal Hub project documentation

Technical guide and user manual

Author: Durim Memishi

Computer Graphics project

Documentation date: 5 November 2026

Implementation baseline: Git commit 14b42a9

Project overview

Limitless Deal Hub is an educational car showroom that combines a React website with interactive Three.js scenes. Visitors can browse 12 vehicles from nine brands, customize a car, compare vehicles and drive a selected design through an animated city. Accounts provide a personal profile, saved designs and a persistent cart.

The site demonstrates model loading, transformations, material editing, lighting, camera control, animation and interaction. Its prices are demonstration values. Checkout creates a demo receipt and does not process payments, place orders or reserve cars. Driving speeds use arcade settings rather than measured vehicle performance.

Live website: https://comp-graph-project.vercel.app/

Source repository: https://github.com/Dm31650/comp-graph-project

Guide contents

1. Project overview and scope

2. Local installation and troubleshooting

3. Visitor and account workflows

4. Three.js rendering and customization

5. City driving and animation

6. Backend API and authentication

7. Database and persistence

8. Deployment and operation

9. Source structure and maintenance

10. Verification and limitations

Current vehicle collection

Porsche 911 GT3 RS, BMW M4 Competition, Mercedes-Benz AMG GT 63, Audi RS7 Performance, Porsche Taycan Turbo S, BMW M8 Competition, Ferrari F8 Tributo, Lamborghini Revuelto, Bugatti Chiron Profilée, McLaren 720S, Pagani Huayra and Ferrari LaFerrari.

Some artist-created models represent a different year or trim from the catalog entry. Each vehicle's model notes and the model credits describe those differences. The project identifies brands independently and does not claim manufacturer affiliation.

Local installation and troubleshooting

Requirements

Use Node.js 24 or newer, npm, Git and VS Code. A recent browser with WebGL support is required for the 3D scenes. Allow enough disk space for dependencies and the locally hosted model files.

Start an existing copy

Open the project folder that contains package.json in VS Code. For the current PC, the working project is at the path below. Run these commands in its PowerShell terminal.

```
Set-Location 'C:\Users\durim\Desktop\comp-graph-2\comp-graph-project-2'
npm ci
npm run dev
```

npm ci installs the locked dependency versions and recreates node_modules. It does not replace project source files. For a fresh copy, clone the GitHub repository into a new directory, then run the same installation and start commands there.

The development launcher starts the account server on 127.0.0.1:3001, waits for its health check and starts Vite on port 5173. Open http://localhost:5173. Keep that terminal running. Ctrl+C stops the launcher. The local database is created automatically at server/data/accounts.sqlite. No separate database application is required.

Verify the backend

```
Invoke-RestMethod http://127.0.0.1:3001/api/health
```

Expected values are service = showroom-accounts and database = ready. npm run dev:client starts only Vite. npm run start:api starts only the local account server. Use npm run dev for normal development because account features require both processes.

Ports already in use

First look for another project terminal and stop its existing development session with Ctrl+C. To identify an unknown listener, inspect the owning process before stopping anything.

```
Get-NetTCPConnection -LocalPort 3001,5173 -State Listen |
  Select-Object LocalAddress,LocalPort,OwningProcess
Get-Process -Id <OwningProcess>
```

Replace the placeholder with the actual number. Stop only a confirmed duplicate project process. Do not stop unrelated applications. Changing API_PORT alone is insufficient because vite.config.js currently proxies /api to port 3001. A custom setup must keep the launcher, proxy and APP_ORIGIN consistent.

If account features fail, check the API health result, the original terminal and the browser's Network panel. If a 3D scene is slow, use Lite graphics in City Drive, close extra 3D tabs and confirm the model request completed. Refresh only after preserving any unsaved work.

Visitor and account workflows

Explore the collection

1. Open the home page and let the animated model load.
2. Browse the brand carousel. It moves automatically and supports arrows, swipe and pause. Hover or keyboard focus pauses movement.
3. Select a brand or use the collection search. Combine the powertrain filter with price or power sorting.
4. Use the heart control to shortlist a vehicle. The shortlist belongs to this browser and device.
5. Select Compare on up to three cards, then use the comparison tray. A comparison URL can identify selected cars directly, such as /compare?cars=7,9,11.

Customize and drive a design

1. Open a vehicle's Details page and move to its customization section.
2. Change Body color, then choose Gloss, Matte or Metallic finish.
3. Choose a graphic design and its accent color if desired.
4. Change Wheel finish separately. The body paint should retain its selected color. If the model needs a manual mapping, choose Select wheels in 3D and pick the rim pieces.
5. Adjust window tint and compare Daylight, Warm and Track lighting.
6. Choose Test drive my design to open the city with that configuration. The generic Test drive link on a collection card opens the vehicle without passing the current studio design.

The customizer automatically stores a draft per vehicle in this browser. A draft is different from a named design saved to an account. Reset replaces that vehicle's current draft with defaults.

Profiles and saved designs

Create an account with a display name, email and matching password confirmation. Passwords must contain 10 to 128 characters and names 2 to 60 characters. After sign-in, the profile shows account information, a photo control and the number of saved designs. Users can edit their name and profile image. The API also retains a signature color field, which the current profile dashboard does not expose as an editing control.

From the customizer, enter a design name and select Save to my profile. The profile's saved collection can preview the design in 3D, reopen it for editing, launch a drive or add a design snapshot to the cart. The server allows up to 100 saved designs per account. Saving an edited configuration creates a saved entry rather than overwriting an existing entry automatically.

Cart and contact behavior

Guest carts use browser storage. Signed-in carts use the database and return when the same account signs in again. Signing out hides that account's cart and switches back to the guest cart. Importing a browser guest cart into an account requires the visible import action. Different configurations of the same car remain separate items, with quantities from 1 to 10 per configuration.

Demo checkout confirms and clears the selected cart and can download a text receipt. Contact Us validates input. With VITE_CONTACT_EMAIL configured, it prepares an email in the visitor's email application, which the visitor must send. Without an inbox, it downloads the inquiry and reports that nothing was sent.

Three.js rendering and customization

Rendering stack

React builds the interface and routes. React Three Fiber connects React components to Three.js. Drei supplies utilities such as orbit controls, environment lighting and HTML overlays. Vite develops and bundles the application. GSAP animates parts of the website interface.

Vehicle metadata lives in src/data/cars.js. A corresponding entry in src/data/carModels.js identifies the local GLB model, display rotation, source author, license and model notes. The browser loads these binary glTF assets from public/models. The newer compressed models use Meshopt decoding through the existing loader setup.

CarViewer and CarModel place the model in a Canvas scene with lighting and a camera. The assembly preparation clones geometry, bakes its world-space position and creates pieces for the animated reveal. Camera controls allow visitors to inspect the model from different angles.

Geometry and transformations

The source scene can contain nested translations, rotations and scales. Applying matrixWorld places each vertex in a common coordinate system before display normalization. Quantized glTF attributes may store coordinates as normalized integers. carAssembly.js converts positions to floating-point attributes before baking and recomputes floating-point normals. This avoids overflow and incorrect surface shading while retaining the source geometry.

Material editing

carCustomization.js maps each supported model's paint, wheel and glass materials to independent roles. A normalized design contains paint, finish, graphic style, accent, wheel color, tint, lighting studio and optional manually selected material targets. Updates preserve the other fields instead of replacing the whole configuration.

Paint changes affect selected body materials. Wheel finish affects the rim materials. Tint affects glass. Gloss, Matte and Metallic adjust the surface response to light. Manual part selection provides a fallback for models with unusual material names. A mesh that combines several physical parts may limit how precisely a user can edit them.

Animated wheels

wheelRig.js detects supported wheel materials and splits combined wheel geometry into four axle-centered assemblies. Each wheel rotates around a pivot as the car moves, and the front pair steers. Interleaved source attributes are copied correctly. The wheel pieces retain the customization materials so changing rims does not repaint the body.

Other graphics features

The Experience page offers Showcase, Details, Compare, My garage and a visual Night drive mode. It includes camera presets, a guided tour and a scene image download. This presentation-style Night drive is distinct from the controllable City Drive game at /test-drive/:id. The account screens also use a rotating wheel sculpture, and the site has an animated 3D background with a pause control.

City driving and animation

Controls

- W accelerates. S slows toward reverse and then drives backward.
- A and D steer left and right. Space brakes, including while throttle is held.
- C cycles camera views. Escape pauses the drive.
- Chase, Front, Left, Right, Overhead and Free views are also available in the interface.
- Free camera supports dragging the scene. Fullscreen depends on browser support.
- Pause, Reset car and Speed straight provide recovery and a quick route to the acceleration corridor.
- Phones and tablets show touch buttons for steering, Go, Reverse and Brake. Landscape gives a wider view.

Simulation loop

TestDrive.jsx gathers keyboard or touch input and updates the vehicle during frame callbacks. drivingPhysics.js divides elapsed time into small simulation steps. Steering returns smoothly to center, resistance slows the vehicle and steering sensitivity reduces at high speed. Collision checks prevent the player from passing through blocked areas. The wheel animation receives speed and steering from this simulation state.

The HUD reports speed in km/h, distance and peak speed. Each car has an arcade speed limit. These settings are part of the game and do not validate catalog performance figures or reproduce a complete vehicle physics model.

Environment and movement

cityLayout.js defines roads and building blocks. The driving scene includes intersections, traffic lights, street details and a roughly 2 km acceleration corridor. CityLife.jsx animates traffic along city loops and pedestrians along sidewalks. Traffic uses the scene's light cycle. These characters provide movement and scale, with simpler behavior than a full traffic simulation.

DriveSky.jsx interpolates the sky, fog, light intensity and reflections between day and night. The transition normally lasts about six seconds. The sun moves downward, the moon rises and stars and illuminated windows appear. A reduced-motion preference shortens the visual transition.

Reliability and performance

The drive pauses and clears held inputs when the window loses focus, becomes hidden or changes orientation. Models load through Suspense, and an error boundary can show a recovery message if a scene fails. Lite graphics reduces scene cost for less powerful devices. The game occupies the page, while browser Fullscreen is a separate optional control.

GPU capability, model size, viewport resolution and network speed affect loading and frame rate. Tests cover several simulation properties, but a physical phone and tablet review remains useful before a formal assessment.

Backend API and authentication

Two runtime implementations

Local development uses server/index.mjs, a Node HTTP server backed by node:sqlite. Vite proxies same-origin /api requests to that server. Production uses api/index.js, which exports server/cloud-handler.mjs as a Vercel function. That handler accesses PostgreSQL through server/postgres.mjs and the pg library.

Both implementations expose the same application routes. The frontend therefore uses relative /api URLs in both environments. Local accounts and cloud accounts belong to separate databases. Starting the local project does not synchronize local accounts to Neon.

API routes

- GET /api/health checks database reachability and returns service and database status.
- GET /api/me returns the current public user or null.
- POST /api/register accepts name, email and password, creates an account and issues a session.
- POST /api/login verifies credentials and issues a session.
- POST /api/logout removes the current session and clears its cookie.
- PATCH /api/profile updates the signed-in user's name, signature color and optional avatar.
- GET /api/cart returns the signed-in user's normalized cart. POST accepts add, remove, quantity, clear or import actions.
- GET /api/designs returns the owner's saved designs. POST saves a named configuration. DELETE removes an entry owned by that user.

Mutating requests must originate from the configured application origin. JSON inputs have a 256 KiB size limit. Typical responses include 400 for invalid input, 401 for missing or invalid authentication, 403 for an invalid origin, 409 for a registration conflict and 429 for too many authentication attempts.

Passwords and sessions

The backend derives password hashes with scrypt and a random salt. It compares login hashes using timingSafeEqual. A successful login creates a random session token. The browser receives the raw token in a showroom_session cookie, while the database stores its SHA-256 digest and an expiry timestamp. Sessions last seven days unless the user logs out earlier.

Production cookies include HttpOnly, Secure and SameSite=Lax. The local HTTP cookie omits Secure for development. The public user object excludes password hashes and salts. Login and registration attempts are limited to 20 within a 15 minute window per client key. Production stores these counters in PostgreSQL, while the local server uses memory.

Access control and boundaries

Protected queries use the authenticated user ID for cart, profile and design ownership. SQL queries use parameters. Production cart and design mutations use transactions and lock the owner's user row to coordinate concurrent changes. These controls support the prototype, but do not substitute for an independent security review. Email verification, password recovery and account deletion flows are not implemented.

Database and persistence

Database records

- users stores id, unique email, display name, password hash, salt, signature color and optional avatar.
- sessions stores a hashed token, user_id and expiry. Each user can have multiple sessions.
- carts stores one record per user with an array of normalized items. Each item identifies a car, a design snapshot and quantity.
- designs stores an entry ID, user_id, car_id, design name, configuration and updated_at value. A user can own many entries.
- login_limits stores the rate-limit key, attempt count and expiry in production only.

The production schema uses foreign keys from sessions, carts and designs to users, with ON DELETE CASCADE. It uses JSONB for cart items and design configurations. Local SQLite stores these JSON values as text and enables foreign keys. Catalog records and model paths remain in source files, rather than database tables.

Where each feature saves data

Account profiles, named saved designs and signed-in carts use SQLite locally or PostgreSQL in deployment. Browser localStorage holds guest carts, per-car customization drafts and the device shortlist and comparison selection. sessionStorage retains a selected test-drive design during that browser tab's session. Clearing browser storage can remove these device-only values without deleting the account's saved entries.

The storage keys include limitless-cart-v1 for guest cart items, car-build-{carId} for customization drafts, limitless-vehicle-shortlist-v1 for shortlist and comparison IDs, and test-drive:{carId} in sessionStorage for the drive configuration.

Schema initialization

The local server creates its required tables at startup. Production ready() initializes its schema inside a transaction and uses a PostgreSQL advisory lock to coordinate concurrent initialization. It retries initialization after a failure. This is initialization logic rather than a versioned migration framework. Later schema changes should use reviewed migrations and a recovery plan.

Backup and account separation

Keep server/data private and out of Git. For a simple local backup, close the development server before copying the database directory, including any SQLite companion files. Restore only into a stopped local instance, and preserve a backup of the previous directory first.

Use the database provider's supported backup or export facilities for Neon. Test recovery before making substantial schema changes. Do not copy credentials or raw account data into documentation, screenshots or GitHub. Local-to-cloud account migration needs a separate reviewed export and import procedure, and is not part of npm run dev.

Deployment and operation

Existing production setup

GitHub main connects to the Vercel project comp-graph-project. Vercel builds the Vite frontend into dist and deploys api/index.js as a function. vercel.json routes supported page URLs to index.html and /api/:route to the backend function. The persistent cloud database is Neon PostgreSQL.

Deployment procedure

1. Verify the local working tree and review the changes. Run the project checks and build.
2. Commit the intended source and assets, then push the branch connected to Vercel. Preserve existing remote history and never force-push simply to resolve a mismatch.
3. In Vercel, confirm the build command is npm run build and the output directory is dist. Use a compatible Node runtime of version 24 or newer.
4. Connect the intended Neon database to the production environment. Configure DATABASE_URL or POSTGRES_URL as a server-only variable. The handler prefers DATABASE_URL when both exist.
5. Configure APP_ORIGIN with the public origin, including https:// and no trailing slash. The handler also accepts the Vercel deployment and production host variables when provided.
6. Redeploy after changing environment variables. Inspect the deployment result before assuming the public alias changed.
7. Open /api/health on the public site. Confirm database is ready. Test sign-in with a controlled account and verify saved designs and cart persistence.

Environment variables

- DATABASE_URL or POSTGRES_URL supplies the production PostgreSQL connection string. Keep it out of source control and never prefix it with VITE_.
- APP_ORIGIN controls allowed mutation origins. Local development defaults to http://localhost:5173.
- API_PORT changes the local account listener, but the Vite proxy must also match.
- NODE_ENV=production enables Secure cookies in the local Node implementation. The deployed cloud handler sets Secure cookies directly.
- VITE_CONTACT_EMAIL is a public frontend inbox address for the contact handoff. It is not a secret or a server email service.

The local Node launcher reads process environment variables. It does not load .env automatically. Vite reads its own frontend environment files. A Vercel environment change affects the deployed function after redeployment, not an already running local Node process.

Common production failures

If the frontend loads but /api/health fails, inspect the function logs, database connection variable and database availability. If writes return 403, check the exact origin and environment assignment. If a route returns 404 after reload, confirm its rewrite in vercel.json. If integration provisioning fails before the build, inspect the Vercel and Neon deployment settings. Preserve the existing production database branch and avoid creating replacement databases as a routine fix.

Source structure and maintenance

Main source locations

- src/App.jsx defines application routes and top-level providers.
- src/components/SiteChrome.jsx supplies the responsive header and footer. BrandCarousel.jsx and CarCollection.jsx implement browsing.
- src/data/cars.js, carModels.js and brands.js define the catalog, model metadata and logo filenames.
- src/components/CarViewer.jsx, CarModel.jsx and carAssembly.js load and prepare 3D vehicles.
- src/components/CarCustomizer.jsx and carCustomization.js manage design controls and material roles. wheelRig.js prepares animated wheels.
- src/pages/TestDrive.jsx, drivingPhysics.js, cityLayout.js, CityLife.jsx and DriveSky.jsx implement the city game.
- src/pages/Account.jsx, ProfileDashboard.jsx and src/auth contain account screens and API clients.
- src/cart contains design normalization, cart state and pricing. src/collection/selection.js handles device shortlist and comparison state.
- server and api contain the local and deployed backend implementations. scripts/dev.mjs starts local services.
- public/models and public/images hold static assets and credit notices. tests contains regression checks.

Supported pages

The main routes are /, /cars/:id, /compare, /experience, /test-drive/:id, /register, /sign-in, /profile, /cart, /about and /contact. The collection is a section on the home page at /#collection. The brand filter can use a brand query parameter.

Adding another vehicle

1. Obtain an appropriately licensed model and preview. Record its author, source, license and modifications. Keep notices for any derivative assets.
2. Add the GLB under a unique public/models folder and the preview under public/images/cars.
3. Add a unique integer ID and catalog record in cars.js. Add matching model metadata in carModels.js.
4. Align the model's length and forward direction with the driving coordinate system. Set its display rotation deliberately.
5. Map paint, wheel and glass roles in carCustomization.js. Add a wheel profile in wheelRig.js if required.
6. If adding a brand, include its original logo, credits and brands.js mapping. Verify the brand filter and carousel.
7. Check customization, wheel independence, saved designs, cart normalization and test driving. Review on desktop and a smaller viewport.
8. Update tests and any documentation that assumes a fixed catalog size. Commit and deploy after the checks pass.

Asset rights

public/models/MODEL-CREDITS.md lists source models and adaptations. public/images/brands/CREDITS.md records logo sources and trademark notices. Licenses differ between models, including noncommercial restrictions on some existing assets. Do not assume every asset permits commercial use. Preserve attribution and model notes when redistributing the project.

Verification and limitations

Repeatable checks

```
node --test tests/*.test.mjs
npm run build
npm run lint
```

The implementation baseline passed 13 automated regression checks and a production build. The tests cover cart actions, distinct design snapshots, invalid data normalization, all 12 catalog assets, support for new cars in carts, shortlist and comparison bounds, customization material reuse, frame-rate consistency, braking, narrow-obstacle collision handling, steering return, quantized geometry and four wheel pivots.

The six new models also passed integration checks for decoding, paint/wheel/glass mappings, four wheel pivots, two front wheels and paint retention after a wheel-finish change. These asset checks were performed during development. The committed regression tests are in tests/. Browser review included all six new cars opening in City Drive, a customized Bugatti retaining its paint and wheel finish, shortlist persistence, comparison and a 390 by 844 viewport review.

Manual acceptance review

Before a presentation, check the home page, brand carousel, all brand filters, comparison and one saved design. Change body color and wheel finish in sequence. Reload the draft. Enter City Drive using Test drive my design, steer, brake, switch cameras and change day to night. Review sign-out and sign-in using the same test account to confirm that the account's cart and saved designs return. Use a separate guest session to avoid confusing guest and account storage.

Current limits and next improvements

The project remains an educational prototype. It has no real checkout, order management or payment processing. Contact handling is an email handoff or download. The game uses simplified physics and scripted road users. Model trim details and manual material selection may vary between source assets. Actual mobile GPU performance needs testing across physical devices.

Future work can add email verification and recovery, reviewed database migrations, API integration tests, broader accessibility checks and asset streaming improvements. Performance work should measure load time and frame rate on named devices before claiming a speed improvement. The existing Three.js bundle produces a build size warning that merits further review.

References

Implementation snapshot: https://github.com/Dm31650/comp-graph-project/tree/14b42a9

Core evidence: package.json, src/App.jsx, src/components/CarCustomizer.jsx, src/components/carAssembly.js, src/components/wheelRig.js, src/pages/drivingPhysics.js, server/index.mjs, server/cloud-handler.mjs, server/postgres.mjs and tests/.

Model and logo credits are available in the repository under public/models/MODEL-CREDITS.md and public/images/brands/CREDITS.md. The live site links the model credits from its footer.